

A verified, complete CAD decision procedure in PVS

An overview of the `pvs_cad` library and the `(cad)` proof strategy

J. Tanner Slagel*

Code and proofs: https://github.com/j-tanner-slagel/pvs_cad

In short. `pvs_cad` is a PVS library, together with a proof strategy `(cad)`, that decides the first-order theory of the real numbers: sentences built from polynomial equations and inequalities with rational coefficients, the connectives AND, OR, NOT, IMPLIES and IFF, and quantifiers FORALL/EXISTS over real variables, written in prenex form (every sentence has one). The procedure is cylindrical algebraic decomposition (CAD), written as an executable PVS function. The library proves it *sound* (an answer is always the truth) and *complete* (it always answers). Inside a proof, `(cad)` evaluates the procedure and turns the answer into a PVS proof that the kernel checks. The whole library, 232 theories and 3,502 formulas, is proved.

What it looks like

Real PVS sessions, with the simplifier's internal messages cut. `(cad)` decides one closed formula where it stands. Four quantifiers, the quadratic formula as an existence theorem:

```
d_quad_suf :
|-----
{1}  FORALL (a: real):
      FORALL (b: real):
        FORALL (c: real):
          EXISTS (x: real):
            a /= 0 AND b ^ 2 >= 4 * a * c IMPLIES
              a * x ^ 2 + b * x + c = 0

Rule? (cad)
Deciding 1 by cylindrical decision, witness first,
Q.E.D.
Real time = 2.109 secs.
```

Motzkin's polynomial [10] is nonnegative, although it is not a sum of squares, so sum-of-squares certificates cannot prove it:

```
b_motzkin :
|-----
{1}  FORALL (x: real):
      FORALL (y: real):
        x ^ 4 * y ^ 2 + x ^ 2 * y ^ 4 - 3 * x ^ 2 * y ^ 2 + 1 >= 0

Rule? (cad)
Deciding 1 by cylindrical decision, witness first,
Q.E.D.
Real time = 1.135 secs.
```

`(cad *)` works on a whole sequent. After `(skeep)` the facts sit in the sequent over skolem constants. The strategy says which formulas it uses and which it skips, which terms become unknown reals, and which sentence

*This report was written with Claude Code using Claude Opus 5.5. The `pvs_cad` library was developed the same way, with Claude Fable 5.1 and then Opus 5 and Opus 5.5; every proof in it is checked by PVS. Claude Code: <https://claude.com/claude-code>.

it decides. Here $\text{member}(x, l)$ is not about reals and is skipped, while $f(x)$ and $\text{length}(l)$ become unknown reals; that $\text{length}(l)$ is nonnegative comes from its type:

```
s_fun :
|-----
{1}  FORALL (x: real, l: list[real]):
      f(x) > x ^ 2 AND member(x, l) AND length(l) = 3 IMPLIES f(x) > -1
Rule? (skeep)
{-1} f(x) > x ^ 2
{-2} member(x, l)
{-3} length(l) = 3
|-----
{1}  f(x) > -1
Rule? (cad *)
cad*: using (-1 -3 1), skipping (-2); unknown reals: cadv_1 = f(x), cadv_2 = x, cadv_3 = length(l)
cad*: deciding FORALL (cadv_1: real): FORALL (cadv_2: real): FORALL (cadv_3: real): (((cadv_3 >= 0))
  ↳ IMPLIES (((cadv_1 > (cadv_2 ^ 2)) AND (cadv_3 = (3))) IMPLIES ((cadv_1 > (-1)))))
Deciding * by cylindrical decision, witness first,
Q.E.D.
Real time = 1.819 secs.
```

A term that is not a polynomial: $\text{sqrt}(x)$ becomes an unknown real, and the fact that it is nonnegative, which comes from its type, is added to the sentence. (cad) alone could not take this goal in any form:

```
s_sqrt :
|-----
{1}  FORALL (x: nreal, a: real):
      a = sqrt(x) AND a * a = x AND a > 1 IMPLIES x > 1
Rule? (skeep)
{-1} a = sqrt(x)
{-2} a * a = x
{-3} a > 1
|-----
{1}  x > 1
Rule? (cad *)
cad*: using (-1 -2 -3 1), skipping nothing; unknown reals: cadv_1 = a, cadv_2 = sqrt(x), cadv_3 = x
cad*: deciding FORALL (cadv_1: real): FORALL (cadv_2: real): FORALL (cadv_3: real): (((cadv_2 >= 0) AND
  ↳ (cadv_3 >= 0)) IMPLIES (((cadv_1 = cadv_2) AND ((cadv_1 * cadv_1) = cadv_3) AND (cadv_1 > (1))) IMPLIES
  ↳ ((cadv_3 > (1)))))
Deciding * by cylindrical decision, witness first,
Q.E.D.
Real time = 2.602 secs.
```

How a proof by (cad) works

The strategy follows the reflection template of Narkawicz, Muñoz and Dutle’s univariate Sturm and Tarski strategies in NASALib [4, 5]. (cad) reads the goal and computes its encoding as data (os , F , ϕ): the quantifier prefix, the polynomials’ coefficient lists and the Boolean skeleton (see “The main theorems”). It then runs `decide5` in PVS’s ground evaluator and instantiates `decide5_correct` with that data. Unfolding `fsem`, the semantics, turns the result back into the original formula: each quantifier is matched, and one reflection lemma per polynomial (`peval_pnorm`) shows that the coefficient list evaluates to the polynomial as written. A FALSE sentence is reported and left in place, and a false hypothesis closes the goal. (cad *) extends this to a whole sequent. It collects the polynomial hypotheses and goals, replaces non-polynomial real terms such as $\text{sqrt}(x)$ or $f(x)$ by universally quantified reals, and decides the implication. The combined formula enters the proof by case, so both branches are checked by PVS.

The trusted base is the PVS kernel plus the ground evaluator that runs `decide5`, the same base as NASALib’s

Sturm and Tarski strategies; beyond that, one trusts the specification: that the semantics `fsem` and the theorem statements (last section) say what is intended. The strategy code (1,957 lines of Lisp) is not trusted: it only chooses proof steps.

What it decides in practice

CAD is doubly exponential in the worst case, so completeness means “always answers, given time”. Some measured times for complete proofs (PVS’s own timing, one M-series Mac):

Sentence	Shape	Time
Wilkinson’s $(x - 1) \cdots (x - 20)$, expanded, is positive for $x < 1$ (coefficients to 2.4×10^{18})	$\forall$, degree 20	2.9 s
Chebyshev T_{10} stays in $[-1, 1]$ on $[-1, 1]$	$\forall$, degree 10	2.0 s
Motzkin’s polynomial is nonnegative (not a sum of squares)	$\forall\forall$, degree 6	1.1 s
Every depressed cubic $x^3 + px + q$ has a real root	$\forall\forall\exists$	0.8 s
Quadratic formula: $a \neq 0 \wedge b^2 \geq 4ac \Rightarrow \exists x : ax^2 + bx + c = 0$	$\forall\forall\forall\exists$	2.1 s
$\forall x \exists y \forall z : z^4 + yz^2 + y \geq x$	$\forall\exists\forall$	22.9 s
No real root of $z^2 + bz + c$ implies $b^2 < 4c$, by $(\text{cad } *)$ on a sequent	quantified hypothesis	1.3 s

The limits show up with three or four variables at degree 3 to 4: AM–GM for three or four variables and Cauchy–Schwarz in the plane do not finish in two minutes. On a set of 100 generated sentences with 1 to 6 quantifiers, 98 were proved within 300 s, and no answer disagreed with Z3.

How it was built

The library was built in stages in September 2026 (273 commits). Most stages also shipped a verified strategy of their own.

1. **Multivariate polynomials.** A normal form for multivariate polynomials over the rationals, proved unique, and reflection lemmas connecting it to PVS real expressions (strategies `mpoly-eq`, `mpoly-simp`).
2. **Real algebraic numbers.** Root isolation, ordering and signs at algebraic numbers, on top of NASALib’s Sturm and Tarski libraries (`alg-roots`).
3. **Branching quantifier elimination.** One-variable quantifier elimination with parametric sign conditions (`qe-exists`, `poly-sign`).
4. **Resultants and subresultants:** determinants, the resultant, and the subresultant theory behind Collins’s projection.
5. **A first cylindrical decision**, `decide`, proved correct for every sentence (`decide_correct`). It was complete but exponential in the number of polynomials: a circle and a line did not finish in 15 minutes.
6. **The fast CAD.** This is the procedure `(cad)` runs. Over each cell of the lower variables, it lifts to the next variable by walking exact rational separators between that variable’s roots. It reads the sign vector at each separator and at each root through Tarski queries at the cell’s sample point, which may be algebraic. This builds one tower of cells per outer sector, for any number of levels. Soundness never assumes what it cannot check: delineability and separation are certified at run time, and a failed check sets ok to FALSE (`decw_correct` for two quantifiers, `decn_correct` for any number).
7. **Completeness.** A review showed that the only way ok could be FALSE was a fixed search limit running out. Some searches were made total; the others share one fuel parameter u . The proof shows that every large enough u makes every check pass. Two facts carry it. First, each search stops at its first success, so a larger u changes nothing once u suffices. Second, the closures that add sign reads draw them from a finite set built level by level from F , so they reach a fixpoint (`decn_complete`). `decide5` raises u until the checks pass; it

terminates on the non-computable measure “least sufficient u ”, which keeps it an ordinary executable PVS function (`decn_u_complete`, then `decide5_decides`).

The library in numbers

PVS theories (files)	232	Formulas proved, whole-library replay	3,502 of 3,502
Lines of PVS specification	18,266	lemmas and theorems	2,388
Lines of strategy code (Lisp)	1,957	TCCs (type-correctness obligations)	1,114
Commits, September 2026	273	Axioms added	none

It builds on NASALib’s Sturm and Tarski (univariate root counting) and on reals, structures, analysis, complex, `mult_poly`, matrices and `interval_arith`.

Where this sits

Tarski showed that the first-order theory of the real numbers is decidable [1], and Collins’s cylindrical algebraic decomposition [2] made the decision practical; this library builds on the PVS [3] libraries of NASALib [4], in particular the univariate Sturm and Tarski decision procedures of Narkawicz, Muñoz and Dutle [5]. A formal correctness proof of CAD exists in Rocq [6], stated for an abstract real closed field; it is a correctness proof, not an executable procedure used as a tactic. Isabelle/HOL has a verified, complete multivariate quantifier elimination procedure [7], exported to executable code; its authors report that without virtual substitution it “appears to hang on all but the simplest univariate examples”. Lean’s Mathlib [8] has, as far as we could find, no CAD formalization and no CAD-based decision tactic, and a Lean project’s roadmap from September 2026 lists CAD, delineability and Sturm theory among what Mathlib lacks [9]; Lean’s nonlinear tactics (`nlinarith`, `polyrith`) are heuristic or check certificates found by an external algebra system, and are not complete. To our knowledge (a survey of September 2026), `pvs_cad` is the first executable, proof-producing CAD that is formally verified sound and complete and is usable as a tactic in an interactive prover.

The main theorems

A sentence is encoded as data: `os` lists the quantifiers outermost first (TRUE for $\forall$, FALSE for $\exists$), `F` lists the polynomials (exact rational coefficients), and `phi` is the Boolean skeleton, whose atoms say “polynomial i has sign s ”. Its meaning is defined by recursion on the prefix, with the real quantifiers of PVS itself:

The semantics (`cad_decide.pvs`)

```
fsem(os, F, phi, ys): RECURSIVE bool =
  CASES os OF
    null: bfeval(phi, F, ys),
    cons(q, r): IF q THEN FORALL x: fsem(r, F, phi, cons(x, ys))
                  ELSE EXISTS x: fsem(r, F, phi, cons(x, ys)) ENDIF
  ENDCASES
```

The decision procedure `decide5` returns a record: `val`, its verdict, and `ok`, a flag that is TRUE when its run-time checks passed. Three theorems say everything about it:

Soundness (`cad_decide5.pvs`)

```
decide5_correct: THEOREM decide5(reverse(os), F, phi)`ok IMPLIES
  (decide5(reverse(os), F, phi)`val IFF fsem(os, F, phi, null))
```

Completeness (complete_all.pvs)

```
decide5_complete: THEOREM decide5(reverse(os), F, phi)`ok  
decide5_decides: THEOREM decide5(reverse(os), F, phi)`val IFF fsem(os, F, phi, null)
```

There are no hypotheses: `os`, `F` and `phi` are arbitrary, so the result covers every closed prenex sentence with any number of quantifiers. `decide5` is a total function, and PVS generated and proved its termination obligations. So the procedure always terminates, always passes its own checks, and always returns the truth of the sentence.

References

- [1] A. Tarski. *A Decision Method for Elementary Algebra and Geometry*. RAND Corporation, 1948; 2nd ed., University of California Press, 1951.
- [2] G. E. Collins. Quantifier elimination for real closed fields by cylindrical algebraic decomposition. In *Automata Theory and Formal Languages*, LNCS 33, pages 134–183. Springer, 1975.
- [3] S. Owre, J. M. Rushby, N. Shankar. PVS: A prototype verification system. In *CADE-11*, LNAI 607. Springer, 1992. <https://pvs.csl.sri.com>
- [4] NASALib, the NASA PVS library collection. <https://github.com/nasa/pvslib>
- [5] A. Narkawicz, C. Muñoz, A. Dutle. Formally-verified decision procedures for univariate polynomial computation based on Sturm’s and Tarski’s theorems. *Journal of Automated Reasoning* 54:285–326, 2015. <https://doi.org/10.1007/s10817-015-9320-x>
- [6] Q. Vermande. Cylindrical algebraic decomposition in Coq/Rocq. In *CPP 2026*, pages 45–58. ACM, 2026. <https://doi.org/10.1145/3779031.3779100>
- [7] K. Kosaian, Y. K. Tan, A. Platzer. A first complete algorithm for real quantifier elimination in Isabelle/HOL. In *CPP 2023*. ACM, 2023. <https://arxiv.org/abs/2209.10978>
- [8] The mathlib Community. The Lean mathematical library. In *CPP 2020*. ACM, 2020.
- [9] Hex project (Lean), roadmap for the real algebraic geometry behind CAD, issue 10300, September 2026. <https://github.com/kim-em/hex-dev/issues/10300>
- [10] T. S. Motzkin. The arithmetic-geometric inequality. In O. Shisha (ed.), *Inequalities*, pages 205–224. Academic Press, 1967.